

Goodreads Recommendation System — Report Sections

Data Preprocessing and Train/Validation/Test Split

CSV-to-Parquet Conversion. The raw interaction file contains approximately 223 million rows. Reading it as CSV on every experiment incurs a full schema-inference pass in addition to the data scan, which is wasteful on a shared cluster. As the first step of the pipeline, we converted the file to Parquet using an explicit schema definition rather than `inferSchema=True`, avoiding the extra scan entirely. All downstream scripts read from the Parquet version, which benefits from columnar storage, predicate pushdown, and compressed I/O.

User-Based Sampling. Before scaling to the full dataset, we prototyped on a 3% user sample. The sample was drawn deterministically: for each row we computed `xxhash64(user_id) % 100` and kept users whose hash value falls below 3. Because `xxhash64` is a pure function of `user_id`, every row belonging to a kept user passes the filter in a single scan with no additional distinct-and-join shuffle. The resulting sample preserves each sampled user’s complete interaction history, maintaining the natural distribution of interaction counts rather than distorting it by downsampling individual records.

Train/Validation/Test Split. Recommendation evaluation requires that each user’s history be divided into an observed portion, used to generate recommendations, and a held-out portion, used to score them. We first discarded users with fewer than five interactions, as they do not provide enough history to support a meaningful three-way split. For each remaining user, interactions were randomly ordered with a fixed seed (`rand(seed=42)`) and then partitioned within each user: the first 60% were assigned to training, the next 20% to validation, and the final 20% to test. Splitting within users rather than across users guarantees that every validation and test user also has a training history, eliminating cold-start cases from the evaluation cohort by construction. All three splits were written to Parquet for reuse across all subsequent experiments.

1. Market Segmentation

For market segmentation, we represented each book by the set of users who read it, using only interactions where `is_read = 1`. To reduce unstable similarities from very rare books, we filtered out books with fewer than 30 readers in the 3% user sample before computing similarity. We then used Spark `MinHashLSH` to efficiently generate candidate book pairs with highly overlapping reader sets.

After MinHash candidate generation, we computed the exact intersection size, union size, and Jaccard similarity for each candidate pair. The final top 100 pairs were sorted by exact Jaccard similarity. The highest-ranked pair had 34 shared readers out of 35 users in the union, giving a similarity of 0.9714. Overall, the top 100 pairs had high reader overlap, with similarities ranging from about 0.97 to 0.86, suggesting that the method identified books occupying similar positions in the reading market.

2. Popularity Baseline

Popularity Score and Recommendation Generation. The popularity baseline recommends the same globally popular books to every user, personalized only by removing books the user has already seen. We defined popularity as the number of training interactions with `is_read = 1` for each book, matching the implicit consumption signal that is the most abundant feedback type in the dataset. This choice also makes the baseline directly comparable to the implicit ALS model, since both use the same behavioral signal.

To generate recommendations, we first selected the top 1,000 globally popular books. Retaining a candidate pool larger than the final top-100 provides headroom: heavy readers may have already seen many popular books, and without extra candidates they could receive fewer than 100 recommendations. For each training user, we removed all books present in their training history via a left-anti join, then retained the top 100 unseen books ranked by global read count. Recommendations were computed once over all training users and cached, so the same precomputed list was reused across both label definitions and both evaluation splits.

Evaluation. We evaluated under two label definitions to make the baseline comparable with later models. The first treats `is_read = 1` as relevance, consistent with the popularity signal. The second treats `rating >= 4` as relevance, matching the preference signal used by the explicit ALS model.

Under `is_read = 1` labels, the baseline achieved MAP 0.0320, NDCG@100 0.1081, precision@100 0.0275, and recall@100 0.1607 on the validation set, and MAP 0.0322, NDCG@100 0.1087, precision@100 0.0276, recall@100 0.1601 on the test set. Under `rating >= 4` labels, validation performance was MAP 0.0341, NDCG@100 0.1071, precision@100 0.0206, recall@100 0.1770, and test performance was MAP 0.0344, NDCG@100 0.1073, precision@100 0.0206, recall@100 0.1754. The close agreement between validation and test scores across both label definitions suggests the baseline is stable and not sensitive to the particular split.

Table 2: Popularity baseline evaluation results.

Table 1: *

Table 1: Top 100 most similar book pairs from the 3% user sample, after filtering to books with at least 30 readers. Pairs are sorted by exact Jaccard similarity. r_1, r_2 are the reader counts of each book; $|\cap|, |\cup|$ are intersection and union sizes of the reader sets.

b_1	b_2	r_1	r_2	$ \cap $	$ \cup $	J	b_1	b_2	r_1	r_2	$ \cap $	$ \cup $	J
3952	3953	35	34	34	35	0.9714	21846	21848	50	54	49	55	0.8909
21738	21739	71	70	69	72	0.9583	17471	17473	52	50	48	54	0.8889
21736	21737	66	67	65	68	0.9559	21840	21843	33	35	32	36	0.8889
3958	3959	44	42	42	44	0.9545	3951	3952	33	35	32	36	0.8889
4383	4385	42	44	42	44	0.9545	21840	21841	33	35	32	36	0.8889
4383	4384	42	42	41	43	0.9535	58991	58992	119	117	111	125	0.8880
58908	58910	41	43	41	43	0.9535	14229	83430	59	58	55	62	0.8871
3955	3957	39	39	38	40	0.9500	8264	8268	96	102	93	105	0.8857
112530	112531	34	36	34	36	0.9444	58921	58922	72	73	68	77	0.8831
207578	207579	32	32	31	33	0.9394	13976	13977	274	289	264	299	0.8829
51199	67248	32	32	31	33	0.9394	56477	56478	32	32	30	34	0.8824
44736	44737	31	33	31	33	0.9394	58934	58935	31	33	30	34	0.8824
12433	12434	31	31	30	32	0.9375	140338	140339	31	33	30	34	0.8824
44669	44673	40	43	40	43	0.9302	113027	113028	39	40	37	42	0.8810
21737	21739	67	70	66	71	0.9296	157997	157998	31	31	29	33	0.8788
58916	58918	52	54	51	55	0.9273	5429	5430	31	31	29	33	0.8788
19320	19321	91	90	87	94	0.9255	3910	3911	152	163	147	168	0.8750
89049	89051	47	47	45	49	0.9184	21847	21848	51	54	49	56	0.8750
21737	21738	67	71	66	72	0.9167	5831	5833	45	45	42	48	0.8750
4385	4386	44	48	44	48	0.9167	4384	4386	42	48	42	48	0.8750
3953	3954	34	35	33	36	0.9167	4383	4386	42	48	42	48	0.8750
21736	21739	66	70	65	71	0.9155	102964	102966	36	39	35	40	0.8750
49272	49273	34	33	32	35	0.9143	4380	4381	36	39	35	40	0.8750
3951	3953	33	34	32	35	0.9143	56480	56482	30	30	28	32	0.8750
3949	3950	32	33	31	34	0.9118	19322	19323	86	77	76	87	0.8736
7277	7278	121	116	113	124	0.9113	21739	21740	70	78	69	79	0.8734
4384	4385	42	44	41	45	0.9111	14473	14474	56	62	55	63	0.8730
56485	56486	51	56	51	56	0.9107	49272	49274	34	39	34	39	0.8718
40051	40052	72	75	70	77	0.9091	42969	42974	56	60	54	62	0.8710
104123	104124	40	44	40	44	0.9091	14226	14227	51	50	47	54	0.8704
15127	15128	62	62	59	65	0.9077	4386	4387	48	53	47	54	0.8704
14118	14119	81	83	78	86	0.9070	15535	15537	231	242	220	253	0.8696
44669	44670	40	42	39	43	0.9070	14030	14119	89	83	80	92	0.8696
58919	58920	71	72	68	75	0.9067	32574	32575	55	59	53	61	0.8689
21846	21847	50	51	48	53	0.9057	40054	40056	92	93	86	99	0.8687
21736	21738	66	71	65	72	0.9028	3953	3956	34	37	33	38	0.8684
3955	3956	39	37	36	40	0.9000	15533	15534	187	196	178	205	0.8683
3956	3957	37	39	36	40	0.9000	42701	42702	262	261	243	280	0.8679
19321	19326	90	94	87	97	0.8969	5682	5684	123	131	118	136	0.8676
11089	79749	46	45	43	48	0.8958	40055	40056	90	93	85	98	0.8673
11810	125309	45	46	43	48	0.8958	39309	39312	117	122	111	128	0.8672
3958	3960	44	47	43	48	0.8958	50992	50993	42	42	39	45	0.8667
21735	21736	61	66	60	67	0.8955	13975	13976	256	274	246	284	0.8662
21732	21733	53	55	51	57	0.8947	14478	14479	88	93	84	97	0.8660
88814	88815	37	35	34	38	0.8947	13977	13978	289	295	271	313	0.8658
3954	3956	35	37	34	38	0.8947	89051	89053	47	50	45	52	0.8654
2352	2353	65	58	58	65	0.8923	89049	89053	47	50	45	52	0.8654
3952	3954	35	35	33	37	0.8919	88813	88815	34	35	32	37	0.8649
69779	69784	125	119	115	129	0.8915	6166	6168	52	58	51	59	0.8644
58909	58910	44	43	41	46	0.8913	27579	59022	259	263	242	280	0.8643

Split	Label	MAP	NDCG@100	P@100	R@100
Val.	is_read=1	0.0320	0.1081	0.0275	0.1607
Test	is_read=1	0.0322	0.1087	0.0276	0.1601
Val.	rating >= 4	0.0341	0.1071	0.0206	0.1770
Test	rating >= 4	0.0344	0.1073	0.0206	0.1754

than simply marking a book as read, so reviewed books are treated as stronger evidence of interest. Interactions where both fields are zero were dropped, as they carry no behavioral signal. Under this scheme, a book that was read but not reviewed receives a score of 1; the review signal contributes an additional 2, so a book that is both read and reviewed receives a score of 3. Any zero-score row is excluded from training entirely.

4. Implicit-Feedback ALS Recommender

Implicit Feedback Signal. For this model we constructed a composite behavioral signal, deliberately excluding numerical ratings. We defined an implicit score as:

$$\text{implicit_score} = \text{is_read} + 2 \times \text{is_reviewed}$$

The coefficient of 2 on `is_reviewed` reflects the intuition that writing a review demands more active engagement

Model Training and Hyperparameter Tuning. We used Spark ALS with `implicitPrefs=True`. In this mode, Spark follows the standard implicit-feedback ALS approach, where positive interactions are treated as preferences and larger interaction values receive higher confidence. In our implementation, the interaction value was `implicit_score = is_read + 2 * is_reviewed`.

We tuned rank and regularization (`regParam`) on the vali-

dation set over ranks $\{5, 20, 50, 100\}$ and regularization values $\{0.01, 0.1, 0.5\}$, while holding $\alpha = 15.0$ and $\text{maxIter} = 5$ fixed. For each configuration, we requested a pool of 500 candidate recommendations per user from ALS, removed books already seen in training via a left-anti join, and scored the remaining top 100 against held-out $\text{is_read} = 1$ labels.

Rank	regParam	NDCG@100	MAP	P@100	R@100
100	0.1	0.1716	0.0568	0.0455	0.2505
100	0.5	0.1707	0.0560	0.0460	0.2481
50	0.5	0.1641	0.0527	0.0447	0.2380
50	0.1	0.1508	0.0482	0.0390	0.2222
50	0.01	0.1494	0.0476	0.0384	0.2214
20	0.5	0.1467	0.0451	0.0385	0.2119
20	0.1	0.1432	0.0444	0.0379	0.2108
20	0.01	0.1398	0.0431	0.0371	0.2063
5	0.1	0.1191	0.0352	0.0320	0.1749
5	0.5	0.1186	0.0350	0.0320	0.1740
5	0.01	0.1174	0.0346	0.0313	0.1733

Table 3: Validation grid for implicit ALS ($\alpha = 15.0$, $\text{maxIter} = 5$). Sorted by NDCG@100.

Rank 100 with $\text{regParam} = 0.1$ was the best configuration, achieving NDCG@100 of 0.1716 on validation. The effect of regularization depended on rank: at ranks 20 and 50, higher regularization (0.5) performed best, while at rank 100 a moderate value (0.1) was optimal. The lowest rank (5) produced the weakest results regardless of regularization, consistent with the expectation that a large, sparse dataset benefits from richer latent representations.

Results and Comparison with the Popularity Baseline. Using the best hyperparameters (rank = 100, regParam = 0.1), the implicit ALS model achieved MAP 0.0568, NDCG@100 0.1716, precision@100 0.0455, and recall@100 0.2505 on validation. On the test set, results were MAP 0.0566, NDCG@100 0.1722, precision@100 0.0458, recall@100 0.2510 — nearly identical to validation, indicating the selected hyperparameters generalize without overfitting to the validation split.

Split	MAP	NDCG@100	P@100	R@100
Val.	0.0568	0.1716	0.0455	0.2505
Test	0.0566	0.1722	0.0458	0.2510

Table 4: Implicit ALS final evaluation (rank = 100, regParam = 0.1, $\alpha = 15.0$).

Compared with the popularity baseline on the test set under the same $\text{is_read} = 1$ labels (MAP 0.0322, NDCG@100 0.1087, precision@100 0.0276, recall@100 0.1601), the implicit ALS model yields substantial gains: approximately 76% relative improvement in MAP, 58% in NDCG@100, 66% in precision@100, and 57% in recall@100. These results confirm that personalization —

even without any explicit preference signal — substantially outperforms a non-personalized global ranking.